

Site Automation

Operations runbooks: health checks, troubleshooting,
ServiceNow, POA&M and patching - with AAP and, later, ACT

Setup guide for new AAP admins | site-automation 0.3.0 | September 2026

What this is. A step-by-step guide to setting up the site's operations runbooks in Ansible Automation Platform: health checks (system, compliance, MariaDB, certificates, patching), troubleshooting, ServiceNow tickets, POA&M status and patching, and later ACT (GenAI). It starts with what each AAP object *is* and walks through every click, saying what each step does, why, and what you should see.

How to read it

New to Ansible or AAP: read chapter 1 first, then do chapter 2 in order. Chapters 3 to 5 are reference: open them when you build a workflow, read a finding, or add ACT.

Rendered from [docs/](#) in the site-automation repository (version 0.3.0). The repository is the source of truth.

Contents

1. Start here: the pieces and how they fit	4
What the runbooks do	4
The ideas behind Ansible, in plain words	4
The AAP objects, and what each one is for	5
How a health check works, start to finish	6
The findings: the one idea that makes everything modular	6
Where things are	7
The order to do things in	7
2. Setting up AAP, step by step	8
Before you start	8
Step 1. Put the repository in your work Git, with your inventory	8
Step 2. Credential types	9
Step 3. Credentials	9
Step 4. Project	10
Step 5. Inventory (filled from the repository)	10
Step 6. The first job template: Health check	10
Step 7. Run it on one host and read the output	11
Step 8. Tune the thresholds	12
Step 9. The other job templates	12
Step 10. ServiceNow	14
Step 11. Workflows and schedules	15
Step 12. Who can do what	15
If something goes wrong	16
3. Workflows and schedules	17
1. Daily health	17
2. Weekly compliance	18
3. Patch with checks	18
4. Fix with approval (ACT)	19
Schedules	20
Things that trip people up	21
4. The runbooks: every check and what to do	22
Health check (playbooks/health_check.yml)	22

Troubleshoot (playbooks/troubleshoot.yml)	29
Certificate report (playbooks/cert_report.yml)	30
POA&M status (playbooks/poam_status.yml)	30
ServiceNow tickets (playbooks/servicenow_tickets.yml)	30
ServiceNow health (playbooks/servicenow_health.yml)	31
Patch hosts (playbooks/patch_hosts.yml)	31
Apply approved ACT fix (playbooks/act_fix_approved.yml)	31
Not built yet (and why)	32
5. Adding ACT (GenAI) later	33
How they fit: checks find WHAT, ACT finds WHY	33
The three levels	33
Turn it on	34
Write your own check (it works with the report, tickets and ACT automatically)	35
Use ACT in a playbook of your own	36
What ACT is told, and what it cannot do	36
Appendix: credential types to paste into AAP	37
ServiceNow API (servicenow_api.yml)	37
MariaDB monitor (mariadb_monitor.yml)	37
Keystore password (keystore_password.yml)	38
STIG Manager API (stigman_api.yml)	38
ACT model key (act_model_key.yml)	38

1. Start here: the pieces and how they fit

This repository holds **runbooks**: jobs that check Linux servers and help troubleshoot them. You run them from **Ansible Automation Platform (AAP)**, by hand or on a schedule. This page explains the few ideas you need. `SETUP_AAP.md` then walks you through the setup, one click at a time.

What the runbooks do

Runbook	What it answers	When it runs
Health check	Is anything wrong on these servers right now? Disk, mounts, services, load/memory, time, network, logging, SELinux, fapolicyd, auditd, accounts, certificates, patching, MariaDB	Daily (the "daily" set) and weekly (the "weekly" set)
Troubleshoot	Someone reports a problem on a host. What does it look like?	On demand
Certificate report	Which certificates expire soon, on every server, in one list?	Weekly
POA&M status	Which POA&M items are overdue or due soon? Which open CAT I/II findings have no POA&M item?	Weekly
ServiceNow tickets	Opens or updates a ticket for each problem a check found. Notes the ones that cleared	After every check (as a workflow step)
ServiceNow health	Is ServiceNow answering? Are the MID Servers up?	Hourly
Patch hosts	Installs updates, reboots if needed, and proves every service came back	Monthly, after an approval
Apply approved ACT fix	Applies exactly the fix a person approved	Only inside a workflow, after an approval

Every check is **read-only**. It looks and reports, and changes nothing. Patching is the only runbook that changes servers, and it waits for an approval first.

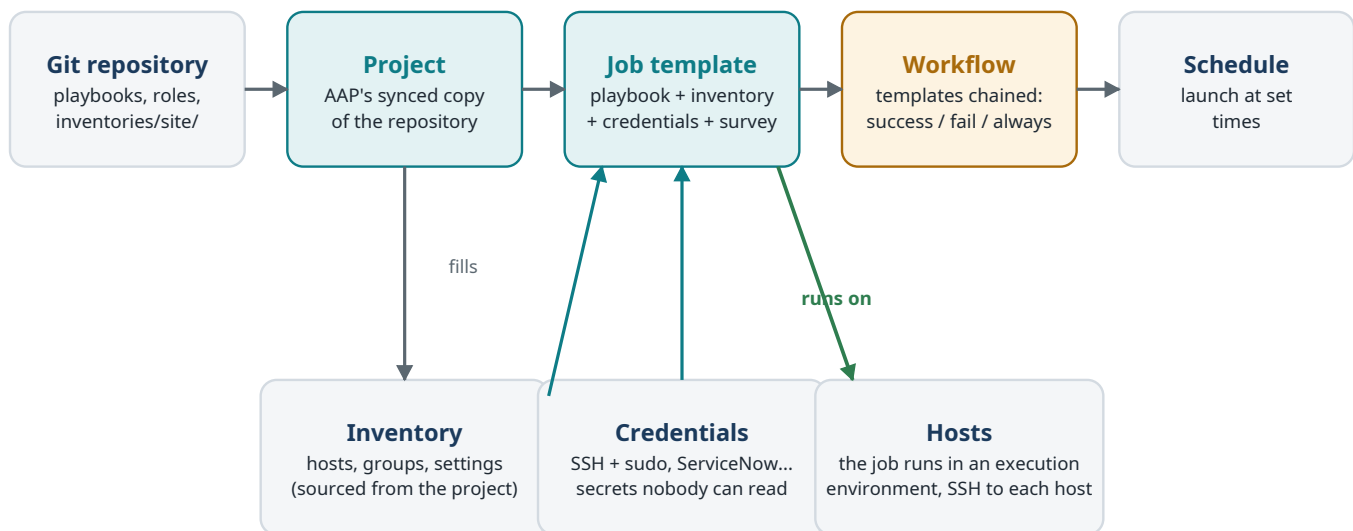
The ideas behind Ansible, in plain words

- **Host**: a server Ansible connects to, over SSH, the way you would.
- **Inventory**: the list of hosts, sorted into **groups** such as `rhel_all` (every Linux server) or `mariadb_hosts`. A group can also carry **settings (variables)**, for example "on the database hosts, the MariaDB container is called servicenow-mariadb".
- **Task**: one step, such as "run `df` and read the result". A **module** is the tool a task uses (`command`, `shell`, `uri`, `set_fact`...).
- **Role**: a folder of tasks that does one job, plus its settings. `roles/check_disk/` is "check the disks". Its settings, with their defaults, are in `roles/check_disk/defaults/main.yml`.
- **Playbook**: the file you run. It says which hosts, and which roles or tasks to run on them. `playbooks/health_check.yml` says "on `rhel_all`, run the checks I was given".

- **become:** run as root through sudo. The checks need it to read root-only information (the audit log, /etc/shadow).
- **Check mode:** a dry run. Ansible shows what it would change and changes nothing. Every runbook here works in check mode, and the checks give the same answers in both modes.
- **Idempotent:** running it twice does no harm. The second run changes nothing.

The AAP objects, and what each one is for

AAP is a web front end that runs playbooks for you. It keeps the passwords, records every run, and runs things on a schedule. You set up a handful of objects once. Each one points at the next:



The AAP objects you create once. The job template is what you launch; a workflow chains templates; a schedule launches either.

AAP object	What it is	Here
Project	A copy of a Git repository that AAP keeps in sync	site-automation (this repository)
Inventory	The hosts and groups, and their settings	Linux servers , filled from inventories/site/ in this repository
Credential type	A form for a kind of secret, and how to hand it to the job	ServiceNow API , MariaDB monitor , ACT model key... (aap/credential_types/)
Credential	One filled-in form: the actual secret. Nobody can read it back	Linux ssh (sudo) , ServiceNow API...
Execution environment	The container a job runs in (Ansible, Python, tools)	The Minimal execution environment is enough. This repository needs no extra collections
Job template	"Run this playbook, on this inventory, with these credentials" plus the questions to ask (survey). This is what you launch	Health check , Troubleshoot , Patch hosts...

AAP object	What it is	Here
Workflow	Job templates chained together. The next step can depend on the last one's result (success, failure, always). It can stop for an approval	Daily health = Health check, then ServiceNow tickets
Schedule	"Launch this template or workflow at these times"	Daily health every day at 06:00

Three more words you will see:

- **Survey:** the questions a template asks when you launch it, such as "which checks?". Each answer becomes a variable for the playbook.
- **Limit:** run on only some hosts or groups, for example `snowdb01.example.mil` or `mariadb_hosts`.
- **Artifacts:** what a job hands to the next step of a workflow. The health check hands its findings to the ServiceNow step this way (`set_stats` in the playbook).

How a health check works, start to finish

1. You click **Launch** on the *Health check* template (or its schedule fires).
2. AAP syncs the project, starts the execution environment, and connects to each host over SSH with the Machine credential.
3. On each host, each chosen check runs read-only commands and turns what it sees into **findings**. One finding is one thing that is wrong, in plain words. It includes a command you can paste to look further:

```
[CRITICAL] disk: /var is 96% full (critical at 95%), 1.2 GiB free of 40.0 GiB
look: timeout 60 du -xh --max-depth=2 /var 2>/dev/null | sort -h | tail -15
```

4. The job prints a summary per host and for the whole fleet. It then **fails the hosts that have findings**. The red status is on purpose: it is how a workflow knows it should open tickets (or ask ACT, or ask for approval).
5. In the *Daily health* workflow, the *ServiceNow tickets* step runs next. It opens a ticket for each finding, or updates the ticket that is already open.

The findings: the one idea that makes everything modular

Every check produces the same kind of finding:

Field	Example	Used by
<code>check</code>	<code>disk</code>	reports, tickets
<code>id</code>	<code>disk:/var</code>	ServiceNow updates the same ticket every run, instead of opening a new one
<code>severity</code>	<code>critical</code> or <code>warning</code>	pass/fail, ticket urgency
<code>summary</code>	<code>/var is 96% full (critical at 95%)</code>	people
<code>hint</code>	<code>du -xh --max-depth=2 /var</code> ...	people, and ACT, which runs it to collect evidence

The report, the ServiceNow tickets and ACT only ever read findings. That is why:

- a new check you write shows up in reports and tickets with no other change;
- ACT works on top of **any** check. Add `use_act=true` and ACT reads the findings and the hint commands' output, then explains the root cause (ADDING_ACT.md).

Where things are

Path	What
<code>playbooks/</code>	the files AAP runs (one per job template)
<code>roles/check_*/</code>	one folder per check. Its settings are in <code>defaults/main.yml</code>
<code>roles/site_findings/</code>	starts, reports and publishes findings, and decides pass/fail
<code>roles/servicenow/</code> , <code>roles/troubleshoot/</code> , <code>roles/patch/</code> , <code>roles/poam/</code>	the other runbooks
<code>roles/site_act/</code>	the bridge to ACT (GenAI)
<code>inventories/example/</code>	a sample inventory and settings. Copy it to <code>inventories/site/</code>
<code>aap/credential_types/</code>	the credential types to create in AAP
<code>poam/poam.csv</code>	your POA&M list (for the POA&M runbook)
<code>docs/</code>	this guide, SETUP_AAP.md, RUNBOOKS.md, WORKFLOWS_AND_SCHEDULES.md, ADDING_ACT.md

The order to do things in

1. **Set up and run one health check by hand** (SETUP_AAP.md, steps 1 to 8). Read its output. Adjust thresholds in the inventory until the findings are the ones you care about.
2. **Add the other job templates** (step 9) and try each once.
3. **Add ServiceNow** (step 10), and build the *Daily health* workflow (WORKFLOWS_AND_SCHEDULES.md).
4. **Schedule** the workflows.
5. **Later: add ACT** to the runbooks you trust (ADDING_ACT.md).

2. Setting up AAP, step by step

Every step says **what** you are doing, **why**, the **clicks**, and what you should **see** when it worked. Menu names are from AAP 2.7 (they are the same in 2.5 and 2.6). If you have not yet, read `START_HERE.md` first. It explains the words used here.

Before you start

You need:

- An AAP account that can create projects, inventories, credentials and templates in your organization (an **Organization admin** does it all).
- A **Linux account on the servers** that AAP logs in as, for example `svc_aap`. It needs an SSH key and sudo. With `NOPASSWD` sudo, add it to `check_accounts_nopasswd_allowed`, or the accounts check will report it.
- A **Git repository at work** that AAP can reach (GitLab, Bitbucket, GitHub Enterprise).
- For ServiceNow: an **API account** (step 10).

Step 1. Put the repository in your work Git, with your inventory

What. Copy this repository into your work Git, and create your real inventory in it. (Already pushed an earlier release at work? Copy the new release's files over your copy, keep your `inventories/site/`, and commit. Then skip to step 2.)

Why. AAP reads playbooks from Git, never from your laptop. The inventory lives in the same repository, so host lists and thresholds are versioned and reviewed like code. Every change has a who, a when and a why.

1. Unpack the release tarball and go into it. Create your inventory from the example:

```
shell
cp -r inventories/example inventories/site
```

2. Edit `inventories/site/hosts.yml`. Replace the example host names with yours and keep the group names: the playbooks use them. A host can be in several groups.
3. Edit `inventories/site/group_vars/all.yml`. These settings apply to every host: the LDAP and SIEM servers each host must reach, the account AAP logs in as (`svc_aap`), and the thresholds. Each setting is explained in `roles/check_<name>/defaults/main.yml`.

4. Push it to your work Git:

```
shell
git init -b main && git add -A && git commit -m "site-automation 0.3.0 + our inventory"
git remote add origin https://gitlab.example.mil/ops/site-automation.git
git push -u origin main
```

5. Note the two values AAP needs in step 4. AAP does not read them from your copy; you type them in once:

```
shell
git remote get-url origin      # the Source control URL
git branch --show-current      # the branch, e.g. main
```

You should see the files in your Git server's web page, including `inventories/site/`.

Step 2. Credential types

What. Teach AAP the kinds of secrets these runbooks use, one form per kind.

Why. AAP ships with types for SSH (Machine) and Git (Source Control). The ServiceNow login and the others are custom. A credential type says which fields the form has and how the secret reaches the job. Every type here passes it as an **environment variable** in the execution environment. The playbook must ask for it on purpose, and it never shows in job output or Git.

For each file in `aap/credential_types/` that you need now:

1. **Automation Execution > Infrastructure > Credential Types > Create credential type.**
2. **Name:** the name at the top of the file (for example `ServiceNow API`).
3. **Input configuration:** paste the `inputs:` part of the file, *without* the `inputs:` line.
4. **Injector configuration:** paste the `injectors:` part, *without* the `injectors:` line.
5. Click **Create credential type**.

File	Name	Needed for
<code>servicenow_api.yml</code>	ServiceNow API	tickets, ServiceNow health (step 10)
<code>mariadb_monitor.yml</code>	MariaDB monitor	the MariaDB check on a containerized database (root login refused there); optional on host installs
<code>keystore_password.yml</code>	Keystore password	optional: certs check on a PKCS12 keystore
<code>stigman_api.yml</code>	STIG Manager API	optional: POA&M cross-check with STIG Manager
<code>act_model_key.yml</code>	ACT model key	later, for ACT (ADDING_ACT.md)

You should see each new type in the Credential Types list.

Step 3. Credentials

What. Store the actual secrets, once.

Why. This is the only place a password or key is kept. Templates point at the credential by name. Nobody, including you, can read the secret back, and it is never in Git or job output.

Automation Execution > Infrastructure > Credentials > Create credential:

Name	Credential type	Fill in
Linux ssh (sudo)	Machine	Username <code>svc_aap</code> , SSH Private Key , Privilege Escalation Method <code>sudo</code> (plus the sudo password if sudo asks for one)
site-automation git	Source Control	only if the repository is private: user name and token, or an SSH key
ServiceNow API	ServiceNow API	instance URL, API user, password (step 10)
MariaDB monitor	MariaDB monitor	needed when MariaDB runs in a container (the ServiceNow database); optional otherwise. See RUNBOOKS.md for the account to ask the DBA for

You should see the credentials listed, with secret fields shown as **ENCRYPTED**.

Step 4. Project

What. Tell AAP where the repository is.

Why. A project is AAP's copy of the repository. Templates pick their playbook from it.

1. **Automation Execution > Projects > Create project.**
2. **Name** `site-automation`. **Organization:** yours.
3. **Execution environment:** **Minimal execution environment** (`ee-minimal`). These runbooks use only modules that come with Ansible itself, so no extra collections or custom execution environment are needed (useful on a disconnected network).
4. **Source control type** `Git`. **Source control URL:** from step 1. **Source control branch/tag/commit:** `main`. Later, pin a release tag such as `v0.3.0`. **Source control credential:** `site-automation git` if the repository is private.
5. Tick **Update revision on launch**. Every job then syncs first, so a change you push is used right away.
6. Click **Create project**.

You should see the project's **Last job status** turn **Successful** after a few seconds. If it fails, open the sync job: it is almost always the URL, the branch name, or the credential.

Step 5. Inventory (filled from the repository)

What. Create the `Linux servers` inventory and fill it from `inventories/site/hosts.yml`.

Why. Sourcing the inventory from the project brings in hosts, groups and the settings in `group_vars/`, all at once, and keeps them in step with Git. You do not type hosts into AAP by hand.

1. **Automation Execution > Infrastructure > Inventories > Create inventory > Create inventory.** **Name** `Linux servers`. Click **Create inventory**.
2. Open it > **Sources** tab > **Create source:**
 - **Name** `site inventory from Git`
 - **Source** `Sourced from a Project`
 - **Project** `site-automation`
 - **Inventory file** `inventories/site/hosts.yml`
 - Tick **Overwrite**, **Overwrite variables** and **Update on launch**.
3. Click **Create source**, then **Launch inventory update**. The rocket icon also starts it.

You should see, under **Groups**, `rhel_all`, `mariadb_hosts`, `patch_hosts`... Under **Hosts** you should see your servers. Open a group: its **Variables** are the ones from `group_vars/<group>.yml`.

Step 6. The first job template: Health check

What. A template that runs `playbooks/health_check.yml`.

Why. A job template is the thing you launch. It fixes which playbook, which inventory and which credentials. Its survey asks which checks to run.

1. **Automation Execution > Templates > Create template > Create job template.**
2. Fill in:

Field	Value	Why
Name	Health check	
Job type	Run	(Check also works: the checks read the same way)
Inventory	Linux servers	
Project	site-automation	
Playbook	playbooks/health_check.yml	
Execution environment	Minimal execution environment	
Credentials	Linux ssh (sudo) (+ MariaDB monitor if you made it)	how AAP logs in
Limit	leave empty, tick Prompt on launch	so you can run it on one host
Privilege escalation	ticked	the checks read root-only files

3. Click **Create job template**.

4. **Survey** tab > **Create survey question**:

- **Question** Which checks?
- **Answer variable name** health_checks
- **Answer type** Multiple Choice (multiple select)
- **Choices**, one per line: daily, weekly, all, disk, mounts, services, performance, time, network, logging, selinux, fapolicyd, auditd, accounts, certs, patching, mariadb
- **Default answer** daily. Tick **Required**. Click **Create question**.

5. Turn the survey **on** (the switch at the top of the Survey tab).

You should see the template in the Templates list with a rocket icon.

Step 7. Run it on one host and read the output

What. Launch the health check on a single server.

Why. Start small. You will see what the checks find on a real host, and whether the thresholds suit you.

1. Click the rocket on **Health check**. **Limit**: one host name. Survey: **daily**. Click **Next**, then **Launch**.

2. The job output scrolls by. The parts to read:

- **Check disk | run, Check services | run...**: each check running (green **ok** lines are normal).
- **Report | result for this host**: the verdict in plain words. Either **HEALTHY**. **Checks run**: ... or a list of **[CRITICAL]** / **[WARNING]** findings. Each finding has a **look**: line: a command you can paste on the host to see more.
- **Report | fleet summary**: one line for all hosts, for example **3 healthy, 1 with findings**.
- A red **FAIL this host because it has findings** at the end. This is **on purpose**: a job with findings fails, so a workflow can react (open tickets). It does not mean the playbook broke.

3. The **Details** tab shows who launched it, when, and the exact revision of the repository.

You should see a finished job, green if the host is healthy, red with findings otherwise.

Step 8. Tune the thresholds

What. Make the findings match what you care about.

Why. Defaults are a starting point. If a warning is expected on some host, change the setting for that host or group, not the code.

1. Change the setting in `inventories/site/group_vars/<group>.yaml` (every host: `all.yaml`). A single host can have `inventories/site/host_vars/<host>.yaml`. Examples:

YAML

```
check_disk_overrides:
  /var/log/audit: {warn: 70, crit: 85}      # this filesystem: warn earlier
check_services_ignore_failed: ['^dnf-makecache']
site_fail_on: [critical]                   # warnings are reported but do not fail the job
```

2. Commit and push. In AAP, run the inventory source again (**Inventories > Linux servers > Sources > Launch inventory update**). A setting in `group_vars` reaches AAP only through the inventory source.
3. Launch the health check again.

You should see the finding gone, or at its new threshold.

Step 9. The other job templates

Create each the same way as step 6: **Inventory** `Linux servers`, **Project** `site-automation`, **Execution environment** `Minimal`. For the ones that log in to hosts, tick **Privilege escalation** and leave **Limit** empty with **Prompt on launch** ticked. Only the differences are listed. A survey question is *Multiple Choice (single select)* unless it says otherwise.

Three templates run on the controller, not on hosts:

POA&M status, ServiceNow tickets and ServiceNow health. For these, do **not** tick Prompt on launch for Limit. A workflow's Limit (for example `patch_hosts`) is passed to every step that prompts for one. These playbooks run on `localhost`, so the Limit would match nothing and the step would silently do nothing, and still show green.

Troubleshoot

On demand, when someone reports a problem. Always launch it with a **Limit**: the host with the problem. Findings do not fail this job.

Field	Value
Playbook	<code>playbooks/troubleshoot.yaml</code>
Credentials	<code>Linux ssh (sudo)</code>
Survey: <code>ts_area</code>	"What kind of problem?" Choices: <code>overview</code> , <code>disk</code> , <code>performance</code> , <code>service</code> , <code>network</code> , <code>selinux</code> , <code>fafolicyd</code> , <code>login</code> , <code>time</code> , <code>logs</code> , <code>mariadb</code> . Default <code>overview</code>
Survey: <code>ts_service</code>	<i>Text</i> , not required. "Service name (for area = service), e.g. <code>nginx</code> "
Survey: <code>ts_target</code>	<i>Text</i> , not required. "host:port it cannot reach (for area = network)"
Survey: <code>ts_since</code>	<i>Text</i> . "How far back to read logs". Default <code>-2h</code>

Certificate report

Weekly: one table of every certificate on every host, soonest expiry first.

Field	Value
Playbook	<code>playbooks/cert_report.yml</code>
Credentials	<code>Linux ssh (sudo)</code> (+ <code>Keystore password</code> if you use it)
Survey: <code>check_certs_warn_days</code>	<i>Integer</i> . "Warn this many days before expiry". Default 30

POA&M status

Weekly. Runs on the controller and reads `poam/poam.csv` from the project.

Field	Value
Playbook	<code>playbooks/poam_status.yml</code>
Credentials	none (+ <code>STIG Manager API</code> for the cross-check)
Privilege escalation	not needed
Limit	empty, and no Prompt on launch (it runs on the controller)
Survey: <code>poam_stigman_api</code>	<i>Text</i> , not required. "STIG Manager API URL (blank = skip the cross-check)"

ServiceNow tickets

Only useful as a workflow step after a check. Runs on the controller.

Field	Value
Playbook	<code>playbooks/servicenow_tickets.yml</code>
Credentials	<code>ServiceNow API</code>
Privilege escalation	not needed
Limit	empty, and no Prompt on launch (it runs on the controller)
Survey	none

ServiceNow health

Hourly. Runs on the controller.

Field	Value
Playbook	<code>playbooks/servicenow_health.yml</code>
Credentials	<code>ServiceNow API</code>
Privilege escalation	not needed
Limit	empty, and no Prompt on launch (it runs on the controller)
Survey	none

Patch hosts

Monthly, inside the *Patch with checks* workflow. Run it as *Check* first: it lists what would be updated and changes nothing.

Field	Value
Playbook	<code>playbooks/patch_hosts.yml</code>
Credentials	<code>Linux ssh (sudo)</code>
Job type	Run, with Prompt on launch ticked, so you can pick <i>Check</i> (a dry run)
Survey: <code>patch_security_only</code>	"Security updates only?" Choices <code>false</code> , <code>true</code> . Default <code>false</code>
Survey: <code>patch_reboot</code>	"Reboot when the updates need it?" Choices <code>when_needed</code> , <code>never</code> . Default <code>when_needed</code>

Apply approved ACT fix

Only inside the approval workflow (ADDING_ACT.md). Create it when you add ACT.

Field	Value
Playbook	<code>playbooks/act_fix_approved.yml</code>
Credentials	<code>Linux ssh (sudo)</code> , <code>ACT model key</code>
Survey	<code>none</code>

Try each one

With a Limit of one host:

- **Troubleshoot:** `ts_area = disk`. Each command's output appears under a `### what it shows` line.
- **Patch hosts:** Job type **Check**. It lists what would be updated and changes nothing.
- **Certificate report:** read the table under `All certificates on all hosts, soonest expiry first`.

Step 10. ServiceNow

What. An API account, its credential, and a first test.

Why. The tickets step and the ServiceNow health check use ServiceNow's REST Table API with a user name and password. No plugin or collection is installed.

1. Ask the ServiceNow admins for an **integration account** (web service access only) with:
 - `itil` (or a role that can create and update incidents);
 - `mid_server` (read the MID Server list), if you want the MID Server check.
2. Create the **ServiceNow API** credential (step 3) with the instance URL, for example `https://servicenow.example.mil`, and the account.
3. In `inventories/site/group_vars/all.yml`, set who gets the tickets:

YAML

```
servicenow_assignment_group: Linux Operations    # a group that exists in ServiceNow
servicenow_min_severity: warning                # or critical: tickets for critical findings only
```

4. Launch **ServiceNow health** once. It proves the account works: **API answers in N s**, then the MID Servers and their status.
5. Build the *Daily health* workflow (WORKFLOWS_AND_SCHEDULES.md). Its tickets step then runs after every check.

Try it safely first: launch the workflow **without** the ServiceNow API credential on the tickets template. The step then only prints **NOT CONNECTED - what would be opened** and sends nothing.

Step 11. Workflows and schedules

See WORKFLOWS_AND_SCHEDULES.md: *Daily health*, *Weekly compliance*, *Patch with checks*, *Fix with approval (ACT)*, and when to schedule each.

Step 12. Who can do what

Open a template or workflow › **User Access** (or **Team Access**) › **Add roles**:

Role	Can
Execute	launch it (and answer its survey)
Approve	approve or deny its approval steps (workflows)
Read	see it and its job output
Admin	change it

For example, give the help desk **Execute** on *Troubleshoot*, and only the server team **Execute** on *Patch hosts* and **Approve** on the patch workflow.

If something goes wrong

You see	It means	Do
Project sync fails	wrong URL, branch or Git credential	open the sync job and read the last lines
UNREACHABLE! ... Permission denied (publickey)	SSH key or user wrong for that host	check the Machine credential; test <code>ssh svc_aap@host</code> from a workstation
Missing sudo password / a password is required	sudo wants a password	put it in the Machine credential (Privilege Escalation Password)
Unknown check(s): ...	a survey choice that is not a check name	fix the survey choices (step 6)
the <name> check could not run: ...	that check hit an error. The others still ran	the message says why, often a missing command such as <code>semanage</code>
skipped: <host> is not in mariadb_hosts	the MariaDB check only runs on database hosts	add the host to <code>mariadb_hosts</code> in the inventory
Findings you do not care about	defaults do not fit that host	step 8: change the setting for its group or host
NOT CONNECTED - what would be opened	the tickets template has no ServiceNow API credential	attach it (step 10)
Tickets step fails: No check results reached this job	the check step before it failed before checking any host, or it was launched on its own	read the first red job in the workflow; run the tickets step only inside a workflow
A controller step (tickets, POA&M, ServiceNow health) shows skipping: no hosts matched	its template prompts for a Limit and the workflow passed one	untick Prompt on launch for Limit on that template (step 9)

3. Workflows and schedules

A **workflow** chains job templates. Each arrow says when the next step runs:

- **Run on success:** only if the step before it succeeded.
- **Run on fail:** only if it failed.
- **Run always:** either way.

The checks *fail on purpose* when they find something. So "Run on fail" means "a problem was found", and "Run always" means "whatever happened". A step also receives what the steps before it published (**artifacts**). That is how the tickets step learns what the health check found.

Build every workflow the same way:

1. **Automation Execution > Templates > Create template > Create workflow job template.**
2. **Name, Organization, Inventory** `Linux servers`. Tick **Prompt on launch** for **Limit**. Leave **Variables** empty: workflow variables would override every step's own answers.
3. **Create workflow job template.** The **workflow visualizer** opens.
4. **Add step > Node type** *Job Template* > pick the template. If it has a survey, the wizard asks for **this step's** answers (for example `health_checks = daily`) > **Finish**.
5. To add the next step: hover over a box > **Add step and link** > pick the template and the **link type** (success / fail / always).
6. **Save**. Closing without saving throws the graph away.

1. Daily health



Daily health: every run hands its findings to the tickets step, healthy or not.

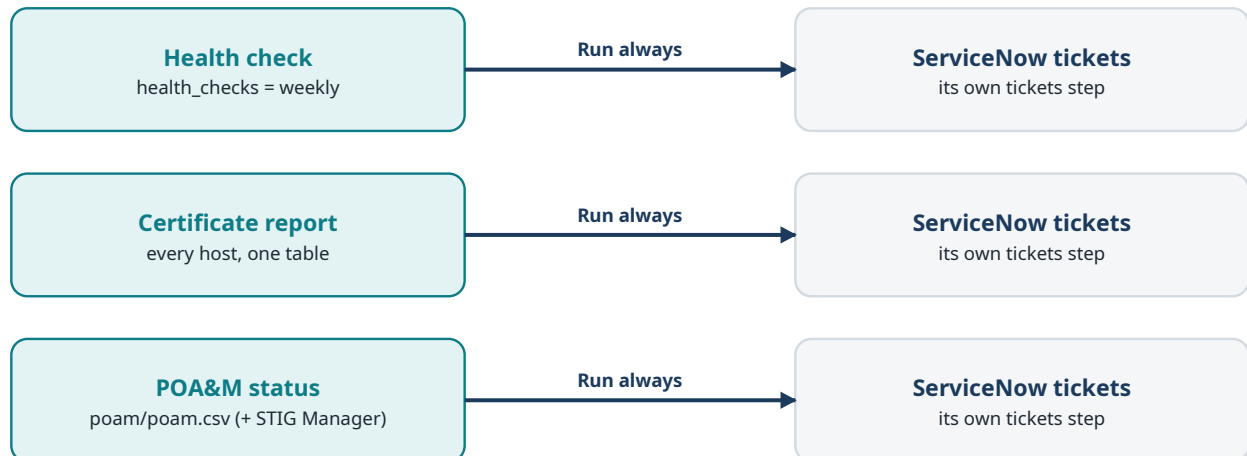
Step	Template	Answers	Link
1	Health check	<code>health_checks = daily</code>	(start)
2	ServiceNow tickets	none	Run always , from step 1

Why "always". The tickets step opens tickets for what failed, and also notes the tickets of problems that cleared. That needs the healthy runs too. It fails when problems exist, so the workflow shows red until they are fixed.

Schedule: every day at 06:00. For faster warning on critical hosts, add an hourly schedule with `health_checks = disk, services, mariadb` and a Limit such as `mariadb_hosts`. The ticket de-duplication keeps that from opening duplicates.

2. Weekly compliance

Three independent chains in one workflow. Add each first box with **Add step** (not "Add step and link"), so they all start together:

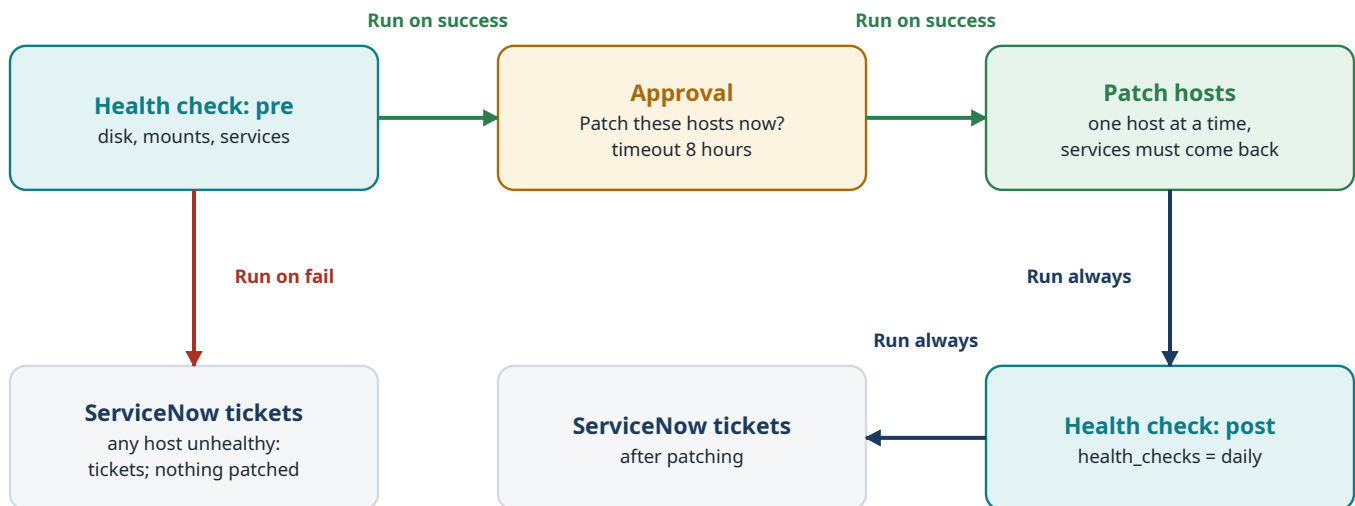


Weekly compliance: three independent chains, each with its own tickets step (a step with two parents would get only one parent's findings).

Give each chain **its own** tickets box. A box with two parents gets only one parent's findings: the artifacts overwrite each other.

Schedule: Mondays at 07:00.

3. Patch with checks



Patch with checks: every host healthy first (one failing host stops the whole run), a person approves, patch one host at a time, then check again.

Step	Template	Answers	Link
1	Health check	health_checks = disk, mounts, services	(start)

Step	Template	Answers	Link
2	Approval	Name <code>Patch these hosts now?</code> , Timeout 8 hours	Run on success , from 1
3	Patch hosts	<code>patch_security_only</code> / <code>patch_reboot</code> as you like	Run on success , from 2
4	Health check	<code>health_checks = daily</code>	Run always , from 3
5	ServiceNow tickets		Run always , from 4
6	ServiceNow tickets		Run on fail , from 1 (tickets for what the pre-check found)

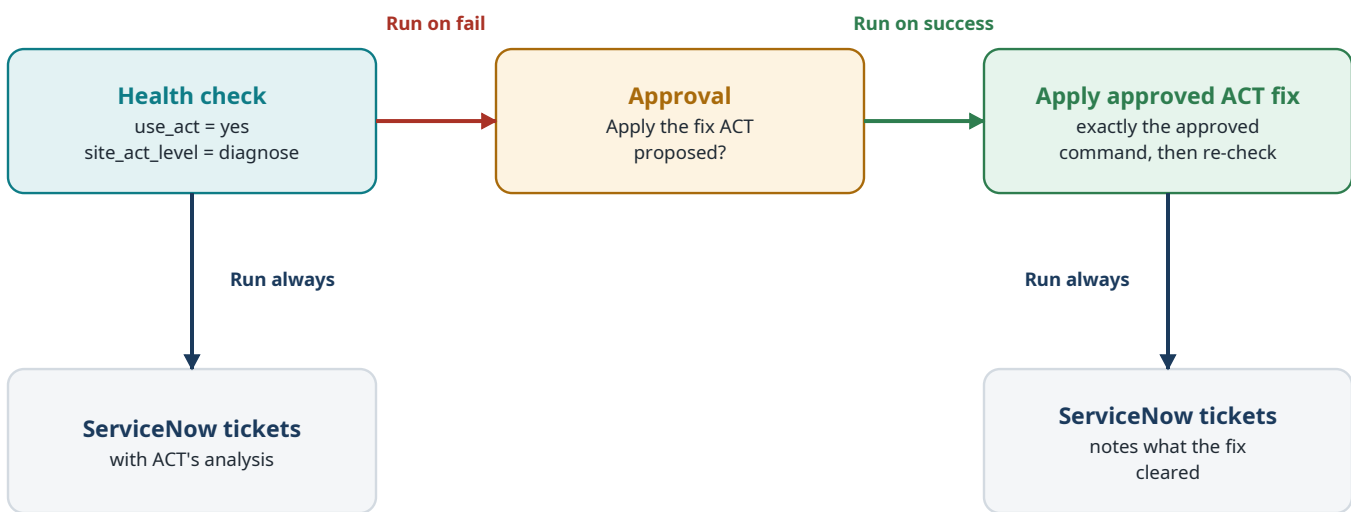
Why these links:

- **Healthy first.** Patching a host that is already failing makes the next problem impossible to diagnose. The pre-check must pass before anyone is asked. **If any one host fails it, the whole run stops before the approval and no host is patched.** Fix that host (the ticket says what is wrong), or launch again with a Limit that leaves it out. Warnings count too while `site_fail_on` includes `warning`. A disk at 86% blocks patching, so clear it before the maintenance window.
- **A person approves.** The approval step waits (up to its **Timeout**) in **Automation Execution** › **Administration** › **Workflow Approvals**. Deny, or no answer, stops the workflow and nothing is changed.
- **Always check after.** The post-check runs whatever happened in patching, so a half-patched host is still reported.

Launch with **Limit** `patch_hosts`, or one group at a time (`stigman_hosts` this week, `servicenow_mid_hosts` next week). **Schedule:** monthly, in your maintenance window, for example the second Wednesday at 20:00. The approval then waits for the on-call person.

4. Fix with approval (ACT)

For later, when you add ACT. See `ADDING_ACT.md` for the three levels.



Fix with approval (ACT): the check job fails with `NEEDS APPROVAL` only when ACT proposed a fix; deny or timeout changes nothing.

Step	Template	Answers	Link
1	Health check (with the ACT survey questions)	<code>use_act = yes, site_act_level = diagnose</code>	(start)
2	ServiceNow tickets		Run always , from 1
3	<i>Approval</i>	Name <code>Apply the fix ACT proposed?</code> , Timeout 30 minutes. Description: <code>Read ACT's report and the NEEDS APPROVAL line in the health check job first.</code>	Run on fail , from 1
4	Apply approved ACT fix		Run on success , from 3
5	ServiceNow tickets		Run always , from 4 (notes the tickets whose problem the fix cleared)

The approver reads, in the health check job, `ACT | what ACT says` (root cause, evidence, the fix) and the red `NEEDS APPROVAL ... ACT proposes: <command>` line. **That exact command** is all the apply step can run. The tickets carry ACT's analysis too.

Schedules

What. A schedule launches a template or workflow at set times.

How. Open the workflow (or template) › **Schedules** tab › **Create schedule**:

1. **Name, Start date/time, Time zone** (use your site's, not UTC, so "06:00" means 06:00).
2. **Repeat frequency** and its details: *Day* (every 1 day), *Week* (on Monday), *Month* (on the second Wednesday)...
3. **Prompts**: the survey answers and the Limit **for this schedule**. Without them, the schedule uses the template's **defaults**. A schedule can only change what the template **prompts on launch** or asks in its survey.
4. **Finish**. The schedule's page shows the next run times. Check them once.

What	When	Launches
Daily health	every day 06:00	workflow <i>Daily health</i>
Quick health (optional)	hourly, Limit <code>mariadb_hosts</code>	workflow <i>Daily health</i> , <code>health_checks</code> = disk, services, mariadb
ServiceNow health	hourly	template <i>ServiceNow health</i> , with an email notification on failure
Weekly compliance	Monday 07:00	workflow <i>Weekly compliance</i>
Patch with checks	monthly, in your window	workflow <i>Patch with checks</i>
Service watch	every 15 minutes	workflow <i>Service watch - approve or self-heal</i> (SERVICE_WATCH_DEMO.md)

Notifications (optional). **Automation Execution › Administration › Notifiers › Create notifier** (type *Email*, your mail relay). Then on a template or workflow › **Notifications** tab › switch on **Failure**. Use it where a ticket cannot help. ServiceNow health is the main case: if ServiceNow is down, it cannot take a ticket about itself.

Things that trip people up

- **Red is not broken.** A check job with findings is red by design. Read `Report | result for this host:` findings mean the playbook worked.
- **Save the visualizer.** Closing without **Save** loses the graph.
- **One tickets box per check box.** Artifacts from two parents overwrite each other.
- **Schedules use survey defaults** unless the schedule sets its own answers (Prompts step).
- **Approvals need a timeout.** Otherwise a forgotten approval keeps the workflow open forever. After the timeout it counts as denied.
- **Workflow variables override node answers.** Leave the workflow's **Variables** empty, and set answers on each step.
- **Limit on the workflow** applies to every step whose template prompts for a Limit. The controller-side templates (ServiceNow tickets, ServiceNow health, POA&M status) must **not** prompt for one, or a workflow Limit makes them match no host and silently do nothing.
- **A tickets step fails when nothing reached it.** If the check step died before checking any host (credential, project sync, execution environment), the tickets step fails too, so a broken schedule shows red instead of green. Read the first red job.

4. The runbooks: every check and what to do

For each runbook: what it answers, its settings (defaults in `roles/<role>/defaults/main.yml`, change them in the inventory), and each finding with what usually fixes it. The `look:` command printed with a finding is always the first thing to run.

Health check (playbooks/health_check.yml)

Runs the checks you pick on each host: a list of names, or the shortcuts:

- `daily` = disk, mounts, services, performance, time, network, logging, mariadb
- `weekly` = selinux, fapolicyd, auditd, accounts, certs, patching
- `all` = both

A host with a finding at a severity in `site_fail_on` (default: critical and warning) fails. That is what a workflow reacts to. A check that itself breaks becomes a warning finding (the `X check could not run: ...`) and the other checks still run.

disk

Finding	Means	Usually
<code>/var is 96% full</code>	local filesystem space (<code>check_disk_warn_pct 85</code> , <code>check_disk_crit_pct 95</code>)	find what grew with the <code>look:</code> command: old logs, core dumps, a runaway log file. <code>journalctl --vacuum-size=500M</code> for the journal
<code>... has used 97% of its inodes</code>	too many small files (space may still be free)	usually a directory with millions of small files (session/cache/spool); <code>look:</code> finds it
<code>/var will be full in about 3.2 days</code>	growth-rate forecast from earlier runs (needs a day of history)	same as above, before it becomes an outage
<code>could not read filesystem usage</code>	<code>df</code> failed or timed out	a hung mount; see the mounts check

Per-mount limits: `check_disk_overrides: {/var/log/audit: {warn: 70, crit: 85}}`. The forecast keeps a small history in `/var/lib/site-health/disk-history.json` (the only file the checks write; a Check-mode run does not write it).

mounts

Finding	Means	Usually
<code>/data (xfs) is in /etc/fstab but is not mounted</code>	it did not mount at boot, or was unmounted	<code>journalctl -b</code> for the mount error; a missing disk or LUN; a typo in fstab
<code>/var is mounted READ-ONLY</code>	the kernel remounted it read-only after disk/filesystem errors	urgent: <code>journalctl -k</code> for I/O errors; storage team; plan an fsck
<code>network mount /mnt/share does not answer within 5s</code>	NFS/CIFS server down, network, or a stale handle	check the server and the network; diagnose only (NetApp is vendor-managed)
<code>network mount ... is 96% full</code>	the share is full	the storage owner

services

Finding	Means	Usually
<code>systemd unit X has FAILED</code>	a unit crashed or exited with an error	<code>systemctl status X</code> , <code>journalctl -u X</code> ; fix the cause, then <code>systemctl restart X</code>
<code>required service X is inactive</code>	a service that must run (<code>check_services_required + _extra</code>) is stopped	same; find out why it stopped before starting it
<code>required service X is not installed</code>	the unit does not exist here	remove it from the list for this group, or install it
<code>required service X is disabled</code>	it will not start after a reboot	<code>systemctl enable X</code>
<code>X has restarted 5 times</code>	a restart loop: it keeps crashing	its log shows the crash

Per group: `check_services_required_extra: [mariadb]`. To ignore a failed unit: `check_services_ignore_failed: ['^dnf-makecache']`.

performance

Finding	Means	Usually
15-minute load is 14.2 on 4 CPUs (3.55 per CPU)	more work than CPUs, for 15 minutes	the top CPU processes are in the finding; a runaway job, or undersized
only 4% of memory is available	memory is nearly exhausted	the top memory processes are in the finding; a leak, or undersized
63% of swap is in use	the host is short of memory	as above
CPUs spend 35% of their time waiting for disk	slow or overloaded storage	<code>iostat -x</code> shows which device; storage latency
the hypervisor takes 15% of CPU time (steal)	the VM host is overcommitted	the virtualization team
the kernel killed processes for lack of memory	OOM kills in the last 24 h (with the victims)	the victim's memory limit, or the host's memory
27 zombie processes	a parent process is not reaping its children	restart the parent (<code>look</code> : shows it)

time

Finding	Means	Usually
the clock is NOT synchronized	chrony uses no time source	<code>chronyc sources -v</code> : unreachable servers, firewall (UDP 123)
chrony is installed but not answering	chronyd is not running	<code>systemctl start chronyd</code>
the clock is 2.4s ahead of NTP time	offset above <code>check_time_offset_warn</code>	chrony corrects it slowly; very large offsets break Kerberos/AD logins at 5 minutes

network

Finding	Means	Usually
there is no default route	the host cannot reach other networks	NetworkManager connection, gateway
this host's own name ... does not resolve	its FQDN is not in DNS or <code>/etc/hosts</code>	DNS record
ldap.example.mil does not resolve (DNS)	a name in <code>check_network_dns_names</code> fails	<code>/etc/resolv.conf</code> , DNS server
LDAP (ldap.example.mil:636) does not answer within 5s	a TCP port in <code>check_network_tcp</code> is unreachable from this host	firewall (local or network), the service down, routing

logging

Finding	Means	Usually
the journal is kept in memory only	<code>/var/log/journal</code> missing: logs vanish at reboot	<code>mkdir /var/log/journal && systemctl restart systemd-journald</code> (or <code>Storage=persistent</code>)
log server SIEM (...) does not answer	forwarding target unreachable	network/firewall; the SIEM collector
rsyslog has 850 MB of undelivered messages queued	the log server has been unreachable for a while	as above; the queue drains once it answers
1203 error messages in the journal	something complains a lot (top senders listed)	look at the top sender first

selinux

Finding	Means	Usually
SELinux is Permissive, it must be enforcing	not enforcing (STIG)	<code>setenforce 1</code> after checking denials; <code>SELINUX=enforcing</code> in <code>/etc/selinux/config</code>
<code>/etc/selinux/config</code> says <code>SELINUX=permissive</code>	a reboot would change the mode	fix the config file
locally added permissive domain(s)	SELinux only logs for those domains	<code>semanage permissive -d <domain></code> once the policy is right
12 SELinux denial(s) since yesterday - top: nginx (httpd_t) denied read on a file labeled user_home_t	SELinux blocked something	a wrong label (fix: <code>restorecon -Rv <path></code>), a boolean (<code>setsebool -P ...</code>), or a real policy need. <code>audit2why</code> says which
SELinux boolean X is off, expected on	a boolean in <code>check_selinux_booleans</code> differs	<code>setsebool -P X on</code>
N file(s) under <code>/etc/nginx</code> have the wrong SELinux label	a dry-run <code>restorecon</code> would relabel them (<code>check_selinux_relabel_paths</code>)	<code>restorecon -Rv <path></code>

fapolicyd

Finding	Means	Usually
fapolicyd is inactive	application allow-listing is off	<code>systemctl start fapolicyd</code> ; its log says why it stopped
fapolicyd is in permissive mode	it logs but blocks nothing	<code>permissive = 0</code> in <code>/etc/fapolicyd/fapolicyd.conf</code>
N file(s) no longer match the fapolicyd trust database	files changed outside RPM	<code>fapolicyd-cli --update</code> , or re-add the file's trust entry
N fapolicyd denial(s) since yesterday - top: /usr/bin/bash -> /opt/app/tool	it blocked programs	a legitimate program needs a trust entry (<code>fapolicyd-cli --file add <path></code>), after review

`check_fapolicyd_required: true` (the example inventory sets it) makes a missing fapolicyd a finding.

auditd

Finding	Means	Usually
auditd is failed	nothing is being audited	<code>systemctl status auditd</code> ; often a full audit log partition
auditing is DISABLED in the kernel	<code>auditctl -s</code> says enabled 0	<code>audit=1</code> on the kernel command line
only 1 audit rule(s) are loaded	the STIG rules are missing or failed to load	<code>augenrules --check</code> , <code>augenrules --load</code> ; a syntax error in <code>/etc/audit/rules.d/</code>
N audit event(s) were LOST	the kernel backlog overflowed	raise <code>-b</code> (backlog) in the rules
the audit log filesystem is 92% full	auditd may stop, or halt the host, when it fills	rotate/offload; check <code>space_left_action</code>
audit records are not forwarded	au-remote is not active (<code>check_auditd_remote_required</code>)	configure audisp-remote

accounts

Finding	Means	Usually
account(s) other than root have UID 0	full root power under another name	investigate first ; remove or renumber the account
account(s) with an EMPTY password	log in without a password	<code>passwd -l <user></code>
3 account(s) not used for 35+ days	inactive and not locked (STIG)	lock them (<code>usermod -L</code>), or list break-glass accounts in <code>check_accounts_ignore_inactive</code>
sudo without a password (NOPASSWD) for: alice	a sudoers entry skips the password	remove it, or allow it on purpose in <code>check_accounts_nopasswd_allowed</code> (e.g. <code>svc_aap</code>)
the password of svc_aap expires in 6 days	an account in <code>check_accounts_expiry_watch</code> expires soon	change it before it breaks automation (and update the Machine credential)

certs

Finding	Means	Usually
certificate CN=... in /etc/stigman/stigman.crt expires in 9 days	a certificate file expires within <code>check_certs_crit_days</code> (critical) / <code>warn_days</code> (warning)	renew it. For STIG Manager: update the <i>TLS certificate</i> credential and run <i>STIG Manager - deploy</i> ; it restarts only nginx
... in STIG Manager (https://127.0.0.1:443) expires in ...	the certificate a TLS endpoint serves	the service's certificate, even if the file on disk was already replaced (restart the service)
... in /opt/.../agent_keystore (alias mid) expires in ...	a Java keystore entry	renew in the keystore (MID Server: ServiceNow's procedure)
could not check the certificate in X	unreadable file, endpoint not answering, or a PKCS12 keystore without its password	fix the path, or add the Keystore password credential

What it looks at: `check_certs_files` (globs), the usual directories (`check_certs_discover_dirs`, skipping CA bundles), `check_certs_endpoints`, `check_certs_keystores`. For a PKCS12 keystore, name the variable that holds the password (`password_env: KEYSTORE_PASSWORD`), never the password.

patching

Finding	Means	Usually
12 security advisories pending, 2 of them rated Critical	updates that fix security issues are waiting	patch (the <i>Patch with checks</i> workflow)
a reboot is needed to finish earlier updates	new kernel or core libraries are not in use yet	reboot in a window
no package has been updated for 60 days	older than <code>check_patching_max_age_days</code>	patch
cannot check for updates	dnf cannot reach its repositories (Satellite, mirror)	repository configuration, network, subscription

mariadb (health only)

Runs on hosts in the `mariadb_hosts` group. It runs status queries only: no data is read and nothing is changed. For MariaDB in a container (the ServiceNow database), set `check_mariadb_container: <name>`; the client inside the container is used.

Finding	Means	Usually
MariaDB is not running	the service or container is down	its log; disk full; a crash
MariaDB is running but the status query failed	cannot log in, or it hangs	the login (see below); <code>max_connections</code> reached
MariaDB restarted 12 minutes ago	a restart nobody may have planned	the error log: crash, OOM kill
143 of 151 connections in use (95%)	clients are about to be refused	the <code>look</code> : query shows which user and client hold them; a connection leak in an app
N client connection(s) were REFUSED	<code>max_connections</code> was reached since the last start	as above; raise <code>max_connections</code> only if the load is real
40 queries are executing at this moment	a pile-up: locking or one slow query	<code>look</code> : lists the oldest
2 query/queries running longer than 300s	long-running queries (user / db / seconds / state; never the query text)	the owner decides; <code>KILL <id></code> only after that
replication (default) is broken: IO thread No	a replica stopped copying	<code>Last_IO_Error</code> / <code>Last_SQL_Error</code> in the finding
replica ... is 1900 seconds behind	replication lag	load on the replica; long transactions on the primary
Galera node is not healthy / cluster has 2 node(s), expected 3	Galera cluster trouble	the node's error log; network between nodes
only 91% of reads come from memory	<code>innodb_buffer_pool_size</code> may be too small	the DBA

Finding	Means	Usually
3 [ERROR] line(s) in the MariaDB log	errors in its log in the last day (latest shown)	read them
the MariaDB data directory is on a filesystem 93% full	MariaDB stops writing when it fills	disk space

Logging in. Without a credential, the check logs in as root through the local socket. That is MariaDB's default for a host install on RHEL (no password needed with sudo). **MariaDB in a container usually gives root a password** (the official images do), and the check then reports *Access denied ... attach the "MariaDB monitor" credential*. For containers, or whenever you prefer not to use root, ask the DBA to create a monitoring account with **no data access**, inside that database with host *localhost*, and attach the *MariaDB monitor* credential to the Health check template:

SQL

```
CREATE USER 'aap_monitor'@'localhost' IDENTIFIED BY '<password>';
GRANT PROCESS, SLAVE MONITOR ON *.* TO 'aap_monitor'@'localhost'; -- MariaDB 10.5.9+
-- older MariaDB: GRANT PROCESS, REPLICATION CLIENT ON *.* TO 'aap_monitor'@'localhost';
```

PROCESS shows other sessions (for long queries); *SLAVE MONITOR* shows replication status. The password reaches the client only through its environment, never its command line.

Troubleshoot (playbooks/troubleshoot.yml)

For "something is wrong on this host": pick the area and the job runs the commands an experienced admin runs first. Each is printed under a line saying what it shows, and the matching health checks run too. Read-only. Findings do not fail this job: you asked for a look, not a verdict.

Area	Commands (what they show)	Checks
overview	uptime, failed units, disk, memory, recent errors, recent reboots	disk, mounts, services, performance, time, logging
disk	space and inodes, biggest directories on the fullest filesystem, deleted-but-open files, files over 1 GiB, journal size	disk, mounts
performance	load, top, vmstat, memory users, disk latency (iostat), load history (sar), OOM kills	performance
service (<i>ts_service</i>)	its status, why it stopped (result, exit code, restarts), its log, dependencies, unit file, listening ports	services (plus that unit)
network (<i>ts_target</i> host:port)	addresses, routes, DNS servers, name lookup, TCP connect, route taken, firewall, connection counts	network (plus that target)
selinux	mode, today's denials, audit2why, changed booleans, local file labels	selinux
fapolicyd	status, today's denials, rules, enforcing?	fapolicyd
login	sshd log, faillock lockouts, failed logins, SSSD domain status, authselect, clock offset	accounts

Area	Commands (what they show)	Checks
time	chrony tracking and sources, timedatectl, configured servers	time
logs	errors, top error senders, rsyslog and its queue, journal size	logging
mariadb	(the MariaDB health check)	mariadb

Add `use_act=true` and ACT reads everything collected and explains the root cause (ADDING_ACT.md).

Certificate report (playbooks/cert_report.yml)

The certs check on every host, and **one table**, soonest expiry first: days left, date, host, subject, and where the certificate is. Hand it to whoever renews certificates. It fails the hosts with a certificate inside `check_certs_warn_days`, so a workflow can open tickets.

POA&M status (playbooks/poam_status.yml)

Reads `poam/poam.csv` (see `poam/README.md` for the columns and how to export from eMASS):

Finding	Means
POA&M 1001 [CAT II]: ... - OVERDUE by 12 days	open item past its scheduled completion date (critical)
POA&M 1002 ...: due in 20 days	due within <code>poam_due_soon_days</code> (30)
... scheduled completion date "TBD" is missing or not a date	fix the date in the list
open CAT I finding V-123456 (...) on 2 asset(s) in collection Production has no POA&M item	STIG Manager cross-check: an open CAT I/II finding that no POA&M item covers

It also lists "maybe ready to close": items whose STIG findings are no longer open in STIG Manager.

The STIG Manager cross-check is optional. It needs a Keycloak client that may use the client-credentials grant, with the scope `stig-manager:collection:read`, and a grant in the collections it reads. **Your realm owner creates that client**; nothing here changes the realm. Then create the *STIG Manager API* credential (token URL `https://<keycloak>/realms/<realm>/protocol/openid-connect/token`, client ID, secret), and set `poam_stigman_api: https://<stigman>/api`.

ServiceNow tickets (playbooks/servicenow_tickets.yml)

A workflow step after a check. For each finding at or above `servicenow_min_severity`:

- **no open ticket** for that problem on that host › it opens an incident: short description [`site-health`] `host: summary`; description with the host, check, finding, the `look:` command, and ACT's analysis if ACT ran; urgency/impact from the severity; assignment group, category and configuration item from the settings.
- **an open ticket exists** › it adds a work note `Still failing at ...` (`servicenow_note_repeats`).
- **a host that was targeted but never reported** › a ticket too (unreachable, login failed).

- **a problem that cleared** (its check ran again and passed) › a work note on its ticket, or it resolves it with `servicenow_resolve_cleared: true`.

Tickets are matched by a correlation ID (`site-health:<host>:<check>:<finding>`, hashed), never by text, so the same problem always updates the same ticket. Without the ServiceNow API credential it only prints what it would open. If no check results reach it at all (the check step died before checking any host), it fails, so a broken schedule shows red instead of green.

ServiceNow health (playbooks/servicenow_health.yml)

From the controller: does the instance answer its REST API, and how fast (`servicenow_slow_seconds`)? Is every MID Server **Up** and **validated**? Hourly is a good schedule. Alert with an AAP **notification** (email) on failure, not with a ticket: a broken ServiceNow cannot take tickets. The MID Server **hosts** (the Linux service, its keystore) are covered by the health check: see `inventories/example/group_vars/servicenow_mid_hosts.yml`.

Patch hosts (playbooks/patch_hosts.yml)

Installs updates with dnf (`patch_security_only` for security fixes only; `patch_exclude` for packages never updated here). Then:

1. **One host at a time** (`patch_serial`). The rollout **stops at the first host that fails**.
2. Refuses with less than `patch_min_free_mb` free in `/var`.
3. Reboots only if `needs-restarting -r` says so (`patch_reboot: when_needed`), and **never** hosts in `patch_never_reboot_groups` (`aap_hosts`). Those get a "reboot by hand" message.
4. After the update (and reboot), **every service that was running before must be running again**. If one is not, the host fails and the rollout stops.
5. Skips hosts in `no_patch` (vendor appliances, and AAP itself in the example inventory), and refuses to patch the machine running the job.

Run it as **Job type: Check** first: it lists what would be updated and changes nothing.

Apply approved ACT fix (playbooks/act_fix_approved.yml)

The step after an approval in the *Fix with approval* (ACT) workflow. It runs **exactly** the commands ACT proposed and a person approved, then re-runs the checks that found the problem. It refuses to run on its own, and skips diagnose-only hosts (`aap_hosts`, `netapp_console_hosts`): there a person applies the fix by hand. See `ADDING_ACT.md`.

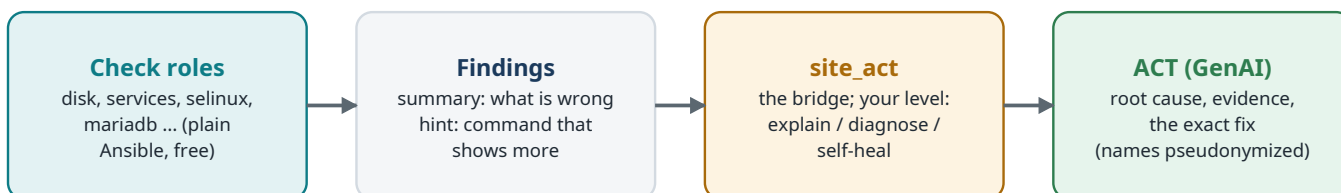
Not built yet (and why)

Idea	Why not yet
SCAP scan › import into STIG Manager	Needs the same Keycloak client as the POA&M cross-check, plus a decision on SCC vs OpenSCAP content. STIG Manager's own importer (stigman-watcher) is the supported path
Certificate renewal workflow	Depends on how your PKI issues certificates. For STIG Manager, renewal already works: update the TLS credential, run <i>STIG Manager - deploy</i>
AAP self-health (job failure rate, execution environment and token age)	Needs an AAP API credential; AAP's own dashboards cover most of it today
Container runbooks (image updates with rollback, drift)	Deferred on purpose; <i>Service watch</i> covers container outages
MariaDB backups, security baseline, password rotation	Out of scope for now: health only, as asked

5. Adding ACT (GenAI) later

The runbooks work without ACT. Add it when the checks are running well and you want the next step done for you: **why** it broke and **what** fixes it. This page shows how the runbooks and ACT fit together, how to turn it on per template, and how to write your own check so it works with ACT automatically.

How they fit: checks find WHAT, ACT finds WHY



Report, ServiceNow tickets and the approval workflow read the same findings - with or without ACT.

Checks find WHAT is wrong; ACT finds WHY. A new check works with ACT automatically through its findings' hints.

- The **checks** decide *whether* something is wrong. They are plain Ansible: the same answer every time, and they are free. The model is only called for hosts with findings.
- Every finding carries a **hint**: the read-only command a person would run next. ACT gets those commands' output as evidence. A new check you write is therefore usable by ACT with no extra work.
- `roles/site_act` is the only place that talks to ACT. Every runbook uses it the same way. Turning ACT on is a variable, not a code change.
- Host names, IP addresses and user names are **pseudonymized** before anything reaches the model (ACT 0.6.18). Its replies are translated back on the host.

The three levels

Level	ACT does	Changes to hosts	Approval step?
explain	reads the evidence (the hint commands' output, collected by Ansible) and writes the root cause and a suggested fix. It runs no commands itself	none	no
diagnose	investigates by running read-only commands itself, then proposes exact fix commands. Every change is refused and waits for a person	only after approval	yes: the job fails with NEEDS APPROVAL › approval step › <i>Apply approved ACT fix</i>
self-heal	like diagnose , but may run fixes that match <code>site_act_allow</code> by itself (for example <code>systemctl restart chronyd</code>). Anything else still waits for approval. The checks then run again to prove the fix worked	only fixes you listed	only for the other fixes

Start with **explain**. Move a runbook to **diagnose** when ACT's explanations have been right for a while. Use **self-heal** only for fixes you would let a junior admin do unasked: start a stopped service, restart a hung

one.

Never fixed automatically, at any level: hosts in `site_act_diagnose_only_groups` (default: `aap_hosts`, `netapp_console_hosts`). There `self-heal` becomes `diagnose`, and the apply step skips them: it prints the approved command for a person to run by hand.

Turn it on

1. The ACT model key credential

Create the credential type `ACT model key` (`aap/credential_types/act_model_key.yml`) and one credential with your GenAI key (`SETUP_AAP.md`, steps 2-3). Attach it to the **Health check** and **Troubleshoot** templates, next to `Linux ssh (sudo)`. This is the only place the key lives.

Non-secret ACT settings go in the inventory:

```
inventories/site/group_vars/all.yml

site_act_env: {GENAI_MODEL: <a model your account offers>}
site_act_extra_instructions: >-
  RHEL 9 servers under DISA STIG. fapolicyd and SELinux are enforcing. Never suggest disabling
  them.
```

2. Two survey questions

On **Health check** and **Troubleshoot** › **Survey** › **Create survey question**:

Question	Variable	Type	Choices	Default
Ask ACT (GenAI) about the findings?	<code>use_act</code>	Multiple Choice (single select)	<code>no, yes</code>	<code>no</code>
What may ACT do?	<code>site_act_level</code>	Multiple Choice (single select)	<code>explain, diagnose, self-heal</code>	<code>explain</code>

3. Try explain by hand

Launch **Troubleshoot** on a host with a known problem: `ts_area = service`, `ts_service = the unit`, `use_act = yes`, `site_act_level = explain`. In the output, read `ACT | what ACT says`. With ACT on, the host's ServiceNow ticket includes this analysis too.

4. diagnose with the approval workflow

Build *Fix with approval (ACT)* (`WORKFLOWS_AND_SCHEDULES.md`). What happens:

1. The health check finds, for example, `required service chronyd.service is inactive`.
2. ACT (`diagnose`) looks at the unit, its log and the config, and proposes `systemctl start chronyd`. The command is refused, so the job fails: `NEEDS APPROVAL - ... ACT proposes: systemctl start chronyd`.
3. The workflow stops at the approval step. The approver reads ACT's report and approves.
4. **Apply approved ACT fix** lets ACT run **exactly** `systemctl start chronyd` (the approved text, escaped, is the only pattern allowed), and ACT verifies it.
5. The same checks run again, without ACT, and the job is green only if the problem is gone. The tickets step notes the ticket as cleared.

5. self-heal for the fixes you trust

```
inventories/site/group_vars/all.yml

site_act_allow:
  - 'systemctl (start|restart) (chronyd|rsyslog|crond)'
```

Each entry is a regular expression that must match the **whole** command. Keep them narrow: name the services, never `.*`. Then pick `site_act_level = self-heal`. After ACT applies an allowed fix, the checks with findings run again. The report shows `cleared: ...` or `still there: ...` and reflects the state **after** the fix.

Write your own check (it works with the report, tickets and ACT automatically)

A check is a role named `check_<name>` that **adds findings** to `site_findings`. That is the whole contract. As an example, here is a check that `/tmp` is not world-writable without the sticky bit.

1. Create `roles/check_tmp/defaults/main.yml`, with its settings and a comment saying what it checks:

```
YAML

---
# TMP: /tmp and /var/tmp must have the sticky bit (mode 1777).
check_tmp_paths: [/tmp, /var/tmp]
```

2. Create `roles/check_tmp/tasks/main.yml`. Collect with read-only commands, and always use `changed_when: false` and `check_mode: false`: read commands then run in a dry run too. Turn what you saw into findings, and add them:

```
YAML

---
- name: tmp | modes
  ansible.builtin.command:
    argv: "{{ ['stat', '-c', '%a %n'] + check_tmp_paths }}"
  register: _tmp
  changed_when: false
  failed_when: false
  check_mode: false

- name: tmp | findings
  ansible.builtin.set_fact:
    _tmp_findings: >-
    {%- set f = [] -%}
    {%- for line in _tmp.stdout_lines if line.split()[0] != '1777' -%}
    {%-   set path = line.split()[1] -%}
    {%-   set _ = f.append({'check': 'tmp', 'id': 'tmp:' ~ path, 'severity': 'warning',
                          'summary': path ~ ' has mode ' ~ line.split()[0] ~ ' (expected
                          1777)',
                          'hint': 'stat ' ~ path ~ '; findmnt ' ~ path}) -%}

    {%- endfor -%}
    {{ f }}

- name: tmp | add them to this host's findings
  ansible.builtin.set_fact:
    site_findings: "{{ site_findings | default([]) + _tmp_findings }}"
```

3. Let the health check know it: in `playbooks/health_check.yml`, add `tmp` to `_health_sets` (to `weekly` and `all`), and add `tmp` to the survey choices of the Health check template.

4. Test it on one host: `ansible-playbook playbooks/health_check.yml -l <host> -e health_checks=tmp`. Then break the condition on purpose (`chmod 777 /tmp` on a test host) and check that the finding appears. A check that never fires proves nothing.

That is all. The report prints it, `set_stats` publishes it, ServiceNow tickets it (the `id` keeps it on one ticket), and with `use_act=yes` ACT receives the summary and runs the `hint` as evidence.

Rules for a good finding:

- **summary**: one line a person understands, with the number and the limit (`/var is 96% full (critical at 95%)`).
- **id**: stable across runs, with no numbers that change (`disk:/var`, not `disk:/var:96`). Otherwise every run opens a new ticket.
- **hint**: **read-only**, runs in under a minute (`timeout 60 ...`), and shows the cause, not just the symptom. ACT runs it as root in `explain` mode, so never put a command that changes something there.
- **severity**: `critical` = act now (outage, security); `warning` = act soon.

Use ACT in a playbook of your own

Any playbook that fills `site_findings` can hand them to ACT the same way:

YAML

```
- name: My check
  hosts: my_hosts
  become: true
  gather_facts: false
  tasks:
    - ansible.builtin.include_role: {name: site_findings, tasks_from: start.yml}
    - ansible.builtin.include_role: {name: site_findings, tasks_from: run_check.yml}
      vars: {_site_check: tmp}                                # runs roles/check_tmp
    - ansible.builtin.include_role: {name: site_act}
      when: use_act | default(false) | bool and site_findings | length > 0
    - ansible.builtin.include_role: {name: site_findings, tasks_from: report.yml}
```

`report.yml` goes last. It prints, publishes (for tickets and the approval workflow), and fails the host when there are findings or a fix waits for approval.

What ACT is told, and what it cannot do

- Its task: the findings (one line each), "find WHY, confirm it with evidence, give the exact fix; several findings may share one cause", plus your `site_act_extra_instructions`.
- **explain**: it runs nothing. Ansible ran the hint commands.
- **diagnose**: only commands ACT can prove are read-only run on their own. Everything else is refused and becomes a proposal.
- **self-heal**: in addition, only commands matching `site_act_allow`.
- Apply step: only the approved commands, character for character.
- Every run is recorded in the AAP job output. The ticket carries ACT's analysis. For the service-watch style incident log, see `SERVICE_WATCH_DEMO.md`.

Appendix: credential types to paste into AAP

Each block below is one file in `aap/credential_types/`. In AAP: **Automation Execution > Infrastructure > Credential Types > Create credential type**. Paste the part under `inputs:` into **Input configuration** and the part under `injectors:` into **Injector configuration** (without those two top-level lines). Copy from the repository file where you can; PDF copy can change spaces.

ServiceNow API (`servicenow_api.yml`)

Used by playbooks/`servicenow_tickets.yml` and playbooks/`servicenow_health.yml` (REST Table API). The same variable names as the certified `servicenow.itsm` collection, so this credential also works if you move to that collection later.

YAML

```
inputs:
  fields:
    - id: instance
      type: string
      label: Instance URL
      help_text: e.g. https://servicenow.example.mil
    - id: username
      type: string
      label: API user name
    - id: password
      type: string
      label: API password
      secret: true
  required: [instance, username, password]
injectors:
  env:
    SN_HOST: "{{ instance }}"
    SN_USERNAME: "{{ username }}"
    SN_PASSWORD: "{{ password }}"
```

MariaDB monitor (`mariadb_monitor.yml`)

For the mariadb health check. Needed when MariaDB runs in a container (the official images give root a password); optional on a host install, where the check can log in as root through the local socket. The password reaches the client only through its environment (`MYSQL_PWD`), never the command line. The account needs no data access at all (`docs/RUNBOOKS.md` shows the GRANT).

YAML

```
inputs:
  fields:
    - id: username
      type: string
      label: Monitor account
    - id: password
      type: string
      label: Password
      secret: true
  required: [username, password]
injectors:
  env:
    MARIADB_MONITOR_USER: "{{ username }}"
    MARIADB_MONITOR_PASSWORD: "{{ password }}"
```

Keystore password (keystore_password.yml)

Optional, for the certs check on a PKCS12 keystore (JKS keystores list without a password). In the inventory, name the variable, never the password: check_certs_keystores: [{path: /path/to/keystore, password_env: KEYSTORE_PASSWORD}]

```
YAML
inputs:
  fields:
    - id: password
      type: string
      label: Keystore password
      secret: true
      required: [password]
  injectors:
    env:
      KEYSTORE_PASSWORD: "{{ password }}"
```

STIG Manager API (stigman_api.yml)

Optional, for the STIG Manager cross-check in playbooks/poam_status.yml. The client is created by your Keycloak realm owner (client-credentials grant, scope stig-manager:collection:read, and a grant in the collections to read) - this repository never changes the realm.

```
YAML
inputs:
  fields:
    - id: token_url
      type: string
      label: Token URL
      help_text: https://keycloak.example.mil/realms/<realm>/protocol/openid-connect/token
    - id: client_id
      type: string
      label: Client ID
    - id: client_secret
      type: string
      label: Client secret
      secret: true
      required: [token_url, client_id, client_secret]
  injectors:
    env:
      STIGMAN_TOKEN_URL: "{{ token_url }}"
      STIGMAN_CLIENT_ID: "{{ client_id }}"
      STIGMAN_CLIENT_SECRET: "{{ client_secret }}"
```

ACT model key (act_model_key.yml)

YAML

```
inputs:
  fields:
    - id: genai_key
      type: string
      label: GenAI API key
      secret: true
    - id: asksage_key
      type: string
      label: AskSage API key
      secret: true
  injectors:
    env:
      GENAI_KEY: "{{ genai_key }}"
      ASKSAGE_API_KEY: "{{ asksage_key }}"
```